

1 · THE TWO ROLES

**Orchestrator**
Plans, writes the Skill.md-shaped goal file, sets acceptance criteria.
Never writes final code.
Fable 5 (effort: high) — or Opus 4.8 as the everyday default

**Executor**
Implements the plan file exactly. Absorbs the token-heavy file reads/writes/diffs. Does not re-plan.
GPT-5.5 (effort: xhigh) via codex — or a Sonnet subagent

Why it saves tokens: the expensive model's context stays small (just the plan, not every file it touches) — the fast executor absorbs the file I/O. You pay premium-model tokens only for the reasoning.

2 · THE 4-STEP LOOP

- 1 Orchestrator writes a goal file — a Skill.md-shaped markdown spec: context, ordered steps, files in/out of scope, and testable "done-when" acceptance criteria (never a vibe — each box must be checkable by reading a diff or running a command).
- 2 Executor implements it verbatim — reads the goal file as its entire brief; does not re-scope or renegotiate criteria.
- 3 Orchestrator verifies output vs. acceptance criteria — reads the diff line-by-line against the done-when checklist. Any unmet box → re-dispatch with the specific gap only, never a fresh goal file.
- 4 Hand off to a fresh chat frequently — /ccc-handoff . Context degrades quality as it fills; frequent small resets beat one giant compaction. Pattern credit: Matt Pocock.

3 · MODEL PAIRING

Orchestrator	Executor	When
Fable 5 (effort: high)	GPT-5.5 (effort: xhigh) via codex	Cross-runtime diversity wanted; codex CLI installed (which codex succeeds)
Opus 4.8 (everyday default)	Sonnet subagent	Stay in one vendor; codex not installed, or single-vendor audit trail preferred
Sonnet/Haiku (cheap, fast)	Opus / Fable (capable)	/ccc-plan-exec : cheap model drafts the loop structure, capable model executes each step's reasoning-heavy body

4 · THE COMMANDS

/ccc-orchestrate — cross-runtime.
Fable/Opus plans → writes a goal file → dispatches GPT-5.5 (via codex) or a Sonnet subagent to execute → verifies the result against the plan's acceptance criteria.

/ccc-plan-exec — Claude-only variant.
Cheap/fast model (Sonnet/Haiku) drafts the plan or loop structure; capable model (Opus/Fable) executes each step. Stay in one runtime.

/ccc-handoff — context reset.
Writes a dense handoff file (same format as /ccc-save-session) to ~/.claude/sessions/, then instructs you to start a fresh chat. Use proactively, not just when a session is dying — especially mid-orchestration.

/ccc-adopt — bring the doctrine to ANY project.
Detects your stack, merges this Orchestrator/Executor section into your project's own CLAUDE.md (delimited block, never clobbers existing content), and tailors a skill/agent pointer list to your stack. Backs up first, previews the diff, re-run safe.

Existing project? Run /ccc-adopt once — merges this doctrine into your CLAUDE.md and tailors skill recommendations to your stack.